File-System Performance

1 Basic Idea

Definition

File-system performance is improved by reducing slow disk accesses and making common access patterns faster.

Why Optimization Matters

Memory access may take about 10 nsec, but a disk access can include 5-10 msec of seek and rotational delay. A small disk read can be about a million times slower than memory.

2 Main Techniques

Technique	Goal
Block cache	Avoid disk reads by keeping recently used disk blocks in memory.
Block read-ahead	Fetch likely next blocks before the program asks for them.
Reduced arm motion	Place related blocks close together on disk.

3 Block Cache

Buffer Cache

A **block cache** or **buffer cache** keeps disk blocks in memory so repeated reads can be served without disk I/O.

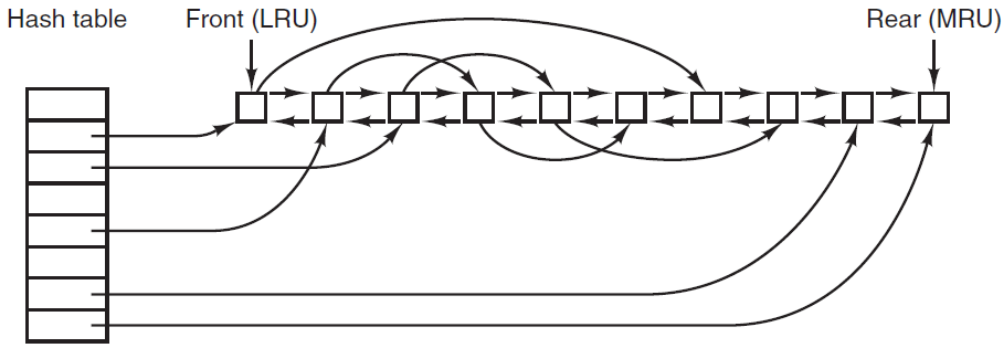

Figure 4-28. The buffer cache data structures.

- On every read, the file system first checks the cache.
- Cache hit: return the block from memory.

- Cache miss: read the block from disk, place it in cache, then return it.
- A hash table maps (device, disk block) to cache entries.
- Collision chains handle multiple blocks with the same hash value.

4 LRU Cache List

LRU

Least recently used replacement keeps old blocks near the front and recently used blocks near the rear.

Action	Result
Block is referenced	Move it to the rear, the MRU end.
Cache is full	Reuse a block near the front, the LRU end.
Dirty block is removed	Write it to disk first.

5 Why Pure LRU Is Not Enough

Metadata Risk

If a dirty i-node, directory block, or indirect block stays in cache too long, a crash can leave the file system inconsistent.

Question	Cache policy effect
Needed again soon?	Keep likely reusable blocks longer.
Critical for consistency?	Write dirty metadata blocks quickly.
Plain data block?	Can usually wait longer than metadata.
Partly full output block?	Keep near MRU because more writes may arrive soon.

6 Write Policy

Policy	Meaning	Trade-off
Periodic write-back	Dirty blocks stay in cache and are flushed by `sync`.	Efficient, but recent data may be lost in a crash.
Write-through	Every modified block is written to disk immediately.	Safer for removable media, but more disk I/O.

UNIX Example

A background process can call `sync` about every 30 seconds, limiting typical crash loss to recent changes.

7 Unified Cache

Single Cache

Some systems combine the buffer cache and page cache, especially when memory-mapped files are supported.

Mapped file pages and cached file blocks both represent file data in memory, so one shared cache avoids duplicate copies.

8 Block Read-Ahead

Read-Ahead

When block k is read, the file system may also start reading block $k + 1$, assuming sequential access.

Access pattern	Read-ahead result
Sequential	Helps; next block may already be in cache.
Random	Hurts; wastes bandwidth and cache space.

Simple Detection

Keep a per-file mode bit: assume sequential at first, clear it after a seek, and set it again if sequential reads resume.

9 Reducing Disk-Arm Motion

Locality

Place blocks likely to be accessed together near each other, preferably in the same cylinder or cylinder group.

- Bitmap allocation makes it easier to choose a free block near the previous block.
- Free lists make close allocation harder unless they track runs of consecutive blocks.
- Clustering allocates groups of consecutive blocks to reduce seeks.
- Rotational placement can also place consecutive blocks favorably within a cylinder.

10 I-Node Placement

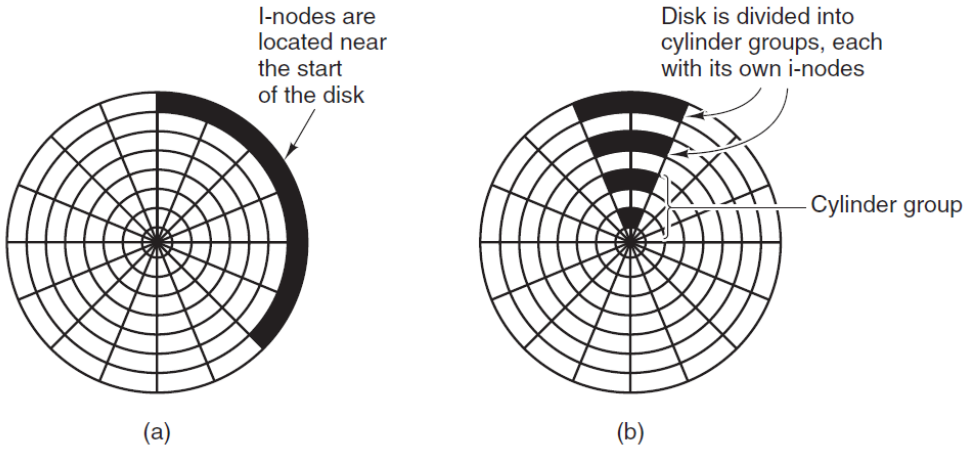

Figure 4-29. (a) I-nodes placed at the start of the disk. (b) Disk divided into cylinder groups, each with its own blocks and i-nodes.

Placement	Effect
All i-nodes at disk start	Short files need one seek for i-node and another for data; average seek is large.
I-nodes near disk middle	Cuts average i-node-to-data seek distance.
Cylinder groups	Each group has its own i-nodes, blocks, and free list; new file data is placed near its i-node.

11 SSDs

Different Device

SSDs have no disk arm or rotational delay, so random access is much closer to sequential access. However, blocks have limited write endurance, so wear leveling becomes important.

12 Final Summary

Summary

File systems improve performance by caching disk blocks, reading ahead for sequential files, and placing related blocks near each other. Metadata must be written carefully because performance optimizations must not destroy consistency.